GNG 5140

Deliverable E: Revised Prototype Analysis and Test Results

Submitted by

Group AI – Coffee Time Saver

Aoqi Liu, 300507810

Lizzy-Preiz, 300457263

Zhe Wang, 300497541

Zhenyan Xing, 300321940

March 25, 2026

University of Ottawa

Abstract

This report presents Deliverable E of Coffee Time Saver, an AI-assisted project management platform designed to help project managers reduce information overload and improve daily coordination. In this stage, the project focuses on two primary user stories: the Daily Project Manager and the Project Risk Checker. Building on the architecture proposed in Deliverable D, the system has evolved from isolated module validation into a fully integrated web application that combines secure authentication, project and task management, document ingestion, daily briefing generation, and risk analysis within a unified interface.

The implemented prototype includes a React/TypeScript frontend, a FastAPI-based backend, PostgreSQL with pgvector, Redis, Celery workers, role-based access control, an LLM gateway, and supporting modules for file processing and email integration. Core frontend pages, including the dashboard, tasks, projects, tools, upload, and settings pages, have been completed to demonstrate the main workflow of the system. Testing at this stage emphasizes functional validation of the full pipeline, with unit tests, integration tests, and frontend functional tests confirming that the major implemented features operate as intended. Key integration bugs identified during development were also resolved.

Overall, this deliverable demonstrates that the proposed platform is technically feasible and sufficiently mature for end-to-end workflow demonstration. The remaining work before Design Day will focus on improving daily briefing quality, validating file extraction and email bot behavior, strengthening the LLM analyzer, and refining the user interface. Future work will address scalability, usability, authentication upgrades, and more rigorous evaluation of LLM quality and local anonymization capabilities for secure deployment contexts.

Table of Contents

Abstract i

Table of Contents ii

List of Figures iv

List of Tables v

1. Introduction 6

1.1 Project Context and System Overview 6

1.2 Problem Relevance (“So What?”) 7

1.3 User Needs and Requirements (“Who Cares?”) 7

1.4 Design Differentiation (“Why This Solution?”) 8

1.5 Report Overview 9

2. Prototype 10

2.1 Architecture 10

2.2 Frontend Prototype 12

2.2.1 Daily Dashboard 12

2.2.2 Tasks 13

2.2.3 Projects Explorer 14

2.2.4 Tools 15

2.2.5 Upload Page 16

2.2.6 Settings 16

3. Prototype and Detailed Testing 20

3.1 Frontend Prototype 20

3.2 Test Execution and Results 22

3.2.1 Unit Tests 22

1.1.1 Integration Tests 24

1.1.2 Frontend Functional Tests 25

1.1.1 Frontend Functional Tests 26

3.3 Remaining Work 26

4. Updated Test and finalization Plan 27

4.1 Current Status and Changes Since Deliverable D 27

4.2 Updated Project Plan 28

4.3 Task Assignment 28

4.4 Contingency Plans 29

1.1.2 Design Day Presentation Risks 29

1.1.1 Development & Testing Risks 30

5. Conclusions and Recommendations for Future Work 30

1.1.2 Status conclusion 30

5.1 Recommendations for Future Work 31

6. Bibliography 33

List of Figures

Figure 1 Architecture flow 10

Figure 2 Frontend Prototype: Daily Dashboard 13

Figure 3 Frontend Prototype: Tasks 14

Figure 4 Frontend Prototype: Projects Explorer 14

Figure 5 Frontend Prototype: Tools 15

Figure 6 Frontend Prototype: Upload 16

Figure 7 Frontend Prototype: Profile Settings 17

Figure 8 Frontend Prototype: LLM Configuration 18

Figure 9 Frontend Prototype: Email Bot Settings 18

Figure 10 Frontend Prototype: Project Visibility 19

Figure 11 Frontend Prototype: User Management 19

Figure 12 Gantt Chart 28

List of Tables

Table 1 Unit test 22

Table 2 Integration Tests 24

Table 3 Frontend Test 25

Table 4 Remainning Work 26

Table 5 Task Assignment 28

1. Introduction

1.1 Project Context and System Overview

Based on the system architecture developed in Deliverable D, this project proposes an AI-powered project management assistant designed to support complex, information-heavy environments such as those encountered by Shared Services Canada (SSC)-like organizations. The system integrates multiple intelligent modules to process fragmented project data and convert it into structured, actionable insights.

The current prototype consists of the following modules:

- **Ingestion Pipeline** – processes unstructured inputs such as emails, documents, and meeting notes into structured artifacts and task lists
- **Priority Ranker** – ranks tasks based on urgency, deadlines, and contextual signals
- **Risk Identifier (Improved)** – detects both explicit and implicit project risks with enhanced reasoning and description quality
- **Daily Planner** – generates structured daily briefs and task execution plans

During this iteration, the **Drift Detection module was removed** due to its high complexity, unresolved accuracy issues, and time constraints within the project timeline. Testing results from Deliverable D showed persistent failures in boundary classification and multilingual reasoning, making it unsuitable for reliable deployment at this stage.

As a result, the project scope has been refined to focus on two primary user stories:

- **Daily Project Manager Support (US1)** – assisting users in organizing, prioritizing, and executing daily tasks

- **Project Risk Checker (US2)** – identifying and communicating potential risks in project workflows

The **Scope Creep Prevention user story (US3)** has been removed from the current implementation. While valuable, it requires a higher level of semantic reasoning and governance reliability than can be achieved within the current constraints.

1.2 Problem Relevance ("So What?")

Project managers in large organizations operate in environments characterized by **high volumes of fragmented information**, including emails, meeting transcripts, reports, and evolving project documentation. This fragmentation leads to:

- increased cognitive load
- missed or delayed identification of risks
- inefficient task prioritization
- reduced situational awareness across the project lifecycle

These challenges directly impact project delivery performance, decision-making quality, and compliance with organizational standards. Addressing this problem is critical because inefficiencies at the project management level can scale into **significant operational and financial consequences** in large systems.

1.3 User Needs and Requirements ("Who Cares?")

The primary users of this system are **project managers and project stakeholders** who require:

- **Centralized visibility** of tasks and project information
- **Automated extraction** of action items from unstructured data

- **Reliable identification of risks**, including those not explicitly stated
- **Dynamic prioritization** of tasks based on changing project conditions
- **Actionable daily planning outputs** that support execution

These requirements reflect the need for a system that reduces manual effort while improving **decision accuracy and operational efficiency**.

1.4 Design Differentiation ("Why This Solution?")

The proposed system differentiates itself from existing tools through its **integrated, multi-module architecture** that combines data ingestion, reasoning, and decision support into a unified workflow. Key advantages include:

- **End-to-end processing pipeline** that transforms raw inputs into actionable outputs
- **Use of LLM-based reasoning** to capture implicit risks and contextual priorities
- **Evidence-based outputs**, improving transparency and trust in system recommendations
- **Modular design**, allowing independent optimization and scalability of each component
- **Balanced deployment strategy**, leveraging both local and cloud models for cost, performance, and security considerations

By focusing on practical functionality and validated performance, this solution aims to bridge the gap between **information overload and actionable project intelligence**.

1.5 Report Overview

This report presents the **revised prototype analysis and testing results** for the current system iteration. It includes:

- updates to the global solution concept
- detailed descriptions of prototype modules and their testing outcomes
- evaluation of system performance against target specifications
- an updated prototype test plan outlining remaining work toward Design Day

The objective of this deliverable is to demonstrate progress from initial prototyping toward a **validated, functional system**, while clearly identifying limitations and next steps for development.

2. Prototype

After consolidating the primary architecture from the last deliverable, the implemented core functionality modules were integrated into endpoints and API services along with an interactive frontend user interface, during this deliverable.

2.1 Architecture

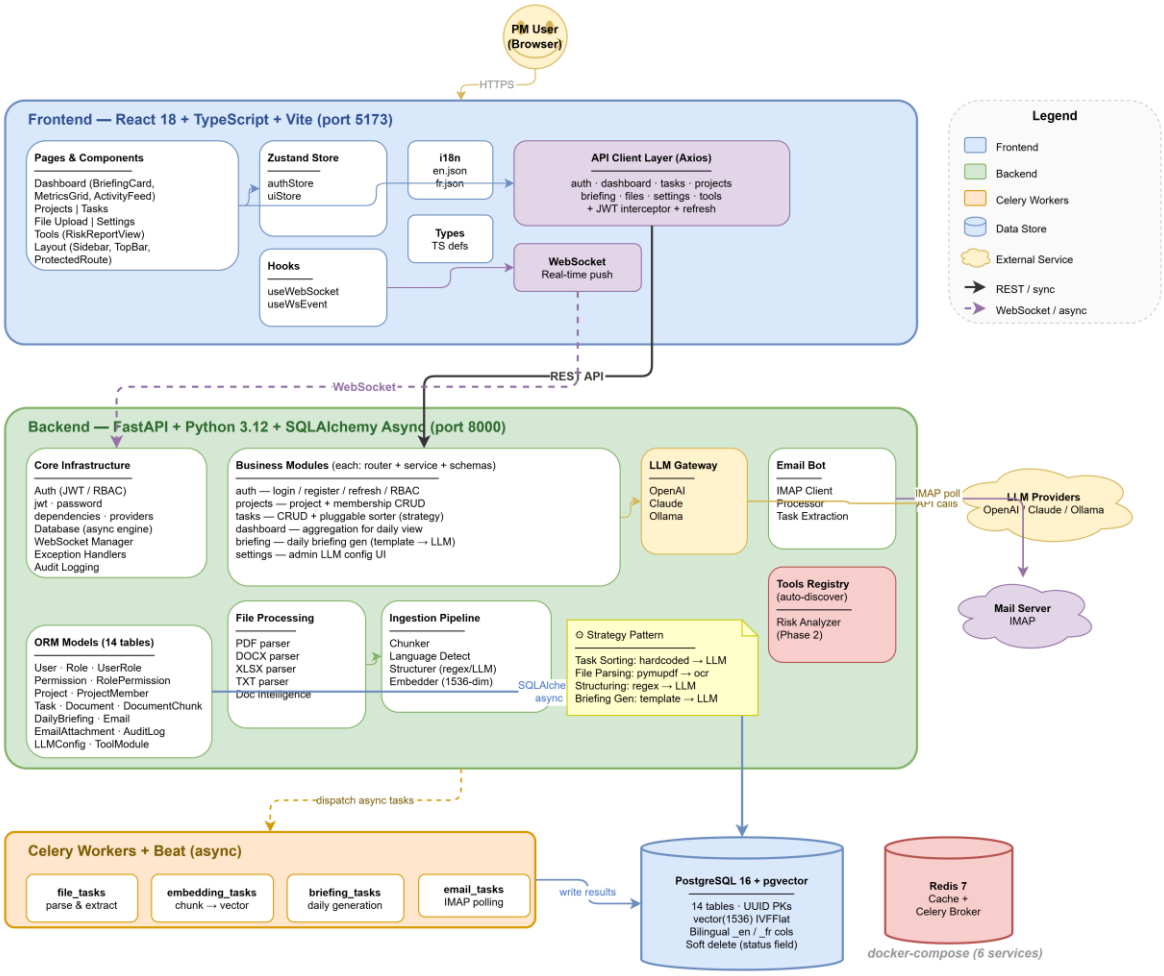

Figure 1 Architecture flow

The introduction of Role-Based Access Control (RBAC) required a far more substantial architectural overhaul than originally anticipated. What began as a lightweight, locally hosted tool for project managers sharing a single information-processing backend has evolved into a full-fledged, multi-service application with proper authentication, authorization, and user management. The need to support JWT-based authentication, per-role permissions, and scoped project membership meant that nearly every layer of the system had to be redesigned, from the database schema (which now spans 14 ORM models including User, Role, Permission, and ProjectMember tables) to the API routing and middleware. Despite this scope expansion, the architecture has been designed with forward compatibility in mind: the current JWT and RBAC implementation provides a clear upgrade path to OAuth 2.0 should the system eventually need to support external identity providers or single sign-on. The modular structure of the system has also been preserved throughout this transition. The AI processing layer, consisting of the Priority Ranker, Risk Engine, Drift Detector, and Daily Planner, remains decoupled from core infrastructure. The ingestion pipeline uses a strategy pattern that allows each processing step (task sorting, file parsing, structuring, and briefing generation) to switch between hardcoded logic and LLM-based approaches independently.

The current stack is organized into six Docker Compose services. The frontend is built with React 18, TypeScript, and Vite, served on port 5173, using Zustand for state management, Axios for API communication, and i18n support for bilingual English and French operation. The backend runs on FastAPI with Python 3.12 and SQLAlchemy Async on port 8000, structured around modular business domains including auth, projects, tasks, dashboard, briefing, and settings, each following a router, service, and schema separation. An LLM Gateway abstracts across OpenAI, Claude, and Ollama providers, while an Email Bot handles IMAP polling and task extraction.

Asynchronous workloads such as file parsing, embedding generation, daily briefing creation, and email polling are dispatched to Celery workers backed by Redis 7, which serves as both cache and message broker. All persistent data resides in PostgreSQL 16 with pgvector enabled for 1536-dimensional vector storage using IVFFlat indexing, supporting bilingual columns and soft-delete semantics across all 14 tables with UUID primary keys.

2.2 Frontend Prototype

2.2.1 Daily Dashboard

The Dashboard serves as the homepage of the web application, including four modules showing an overview of the daily task content. This page begins with a Daily Briefing module, which provides a summarized list of tasks pending to do. Following the text summarization of the tasks, the Key Metrics module displays a straightforward breakdown of the current tasks status, including the number of active projects, overdue tasks, open tasks, files processed today and unread emails. Below the overview section, a list of recent activities and a simple portal for adding task documents are designed to offer immediate access to task management tools and highlight recently processed tasks.

This dashboard page aligns with the user story which requires the tool to automatically generate task lists for the project manager to quickly understand what needs to be done and what happened previously.

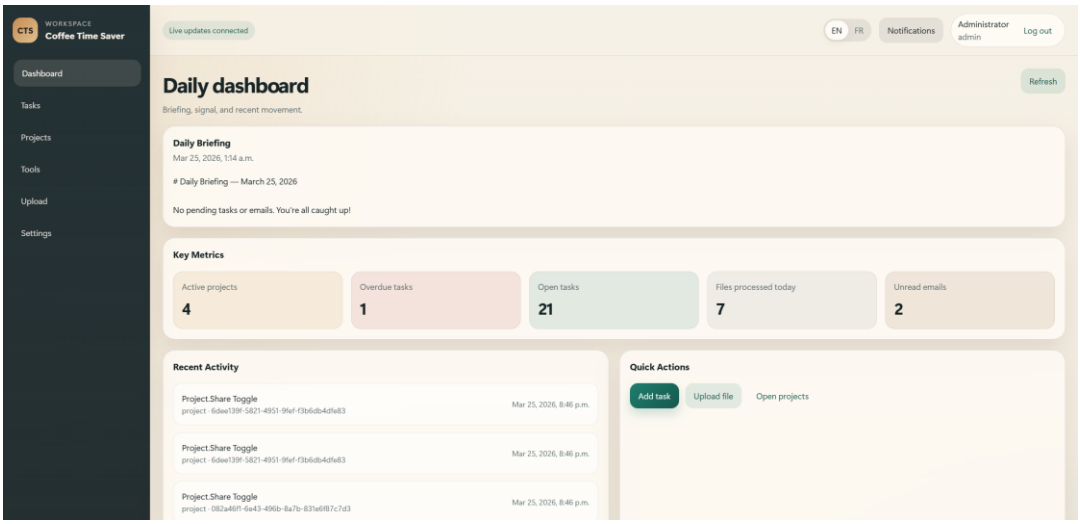

Figure 2 Frontend Prototype: Daily Dashboard

2.2.2 Tasks

The Tasks page offers the portal to manually input task information into the system, including a task's title, description, priority, due date, and its dependent project. In addition, this page acts as a task explorer with filtering options for the user to browse the tasks stored in the database, along with the task information and task status.

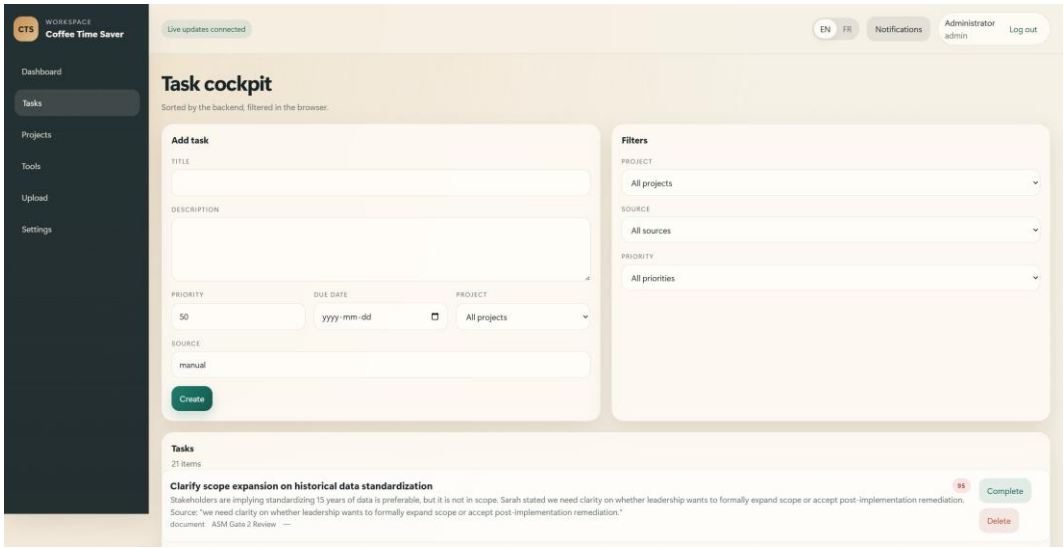

Figure 3 Frontend Prototype: Tasks

2.2.3 Projects Explorer

The Project Explorer page is similar to the Tasks page, allowing the user to manually input project information and browser all projects in the database.

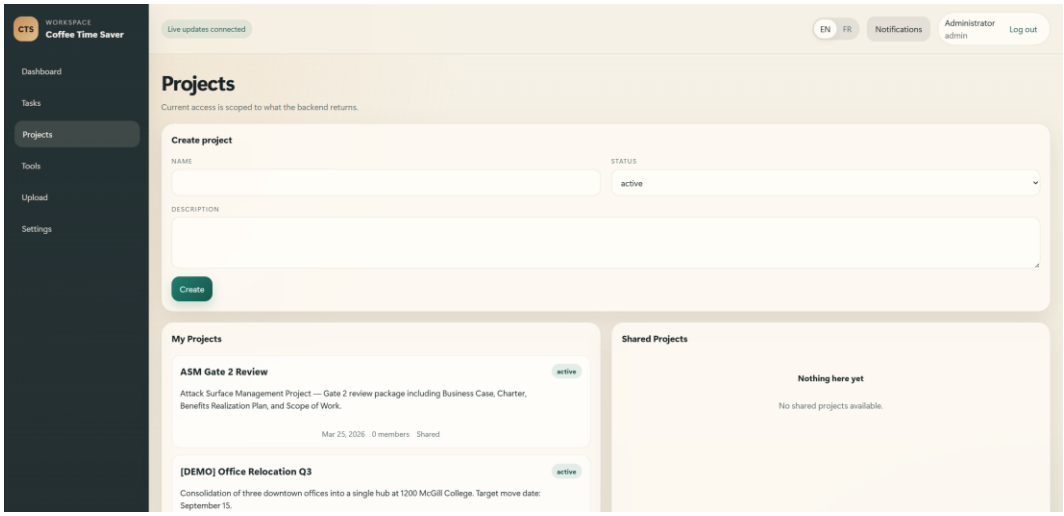

Figure 4 Frontend Prototype: Projects Explorer

2.2.4 Tools

The Tools page offers the user an access panel for manually running the backend services for specific projects. For this deliverable, the project risk analyzer is implemented in the frontend, which is used to identify risks for specific projects by processing available documents in the system.

The Project Risk Analyzer aligns with the user requirement for building a project risk checker, where the project managers expect a system that can automatically detect inconsistencies between project dependencies and descriptions in order to identify potential risks. This feature allows the users to directly access the core functionality, providing verification options and transparency in how the system operates.

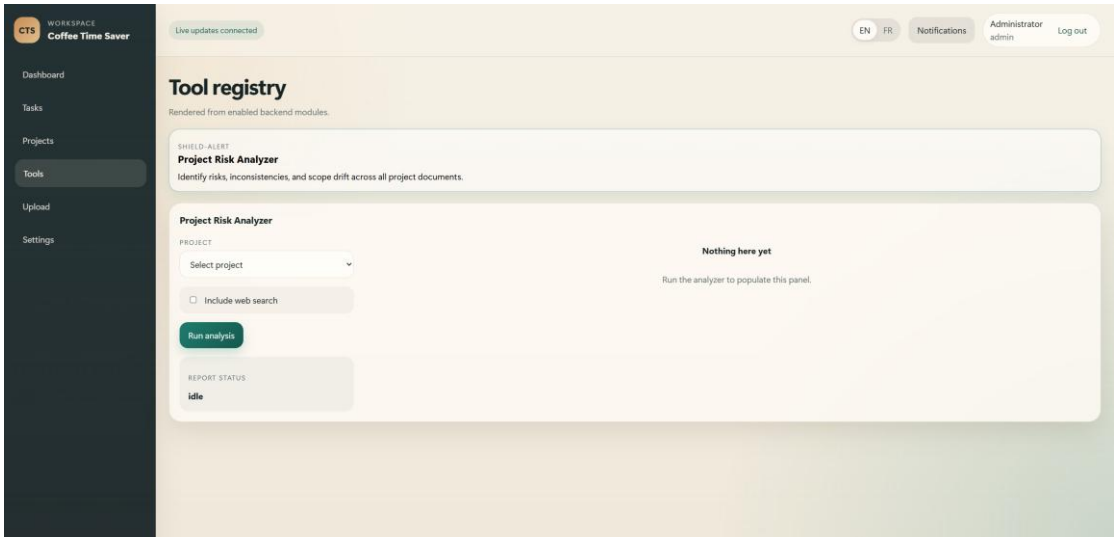

Figure 5 Frontend Prototype: Tools

2.2.5 Upload Page

The Upload page allows users to manually submit files related to specific projects into the system, with a specified project name and document type. A list of the uploaded files can be found in the module under the uploading block.

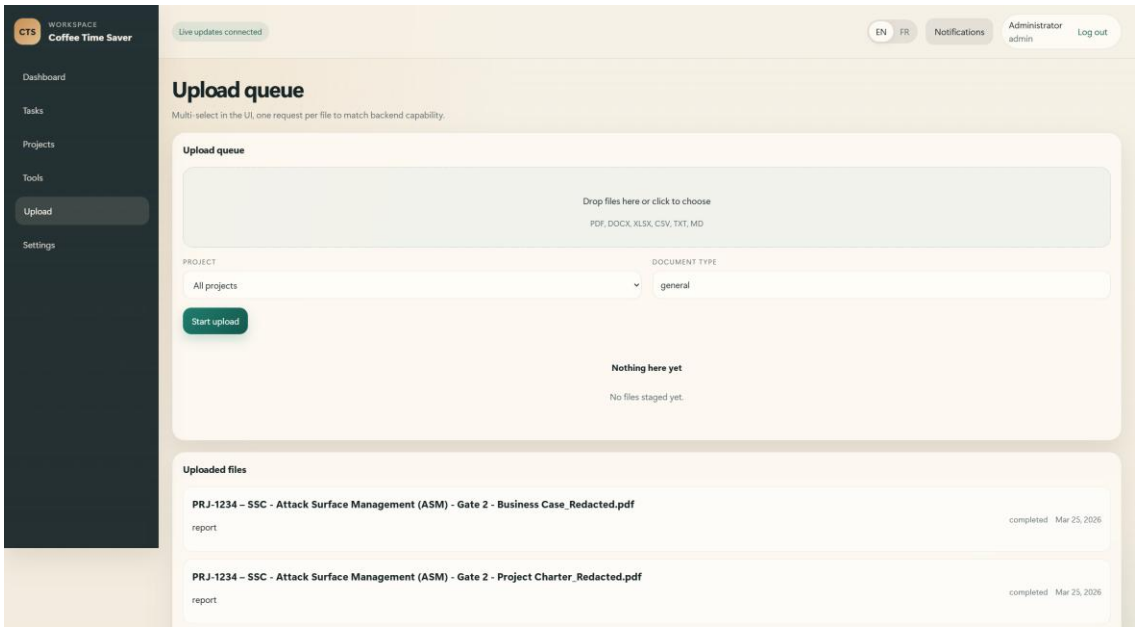

Figure 6 Frontend Prototype: Upload

2.2.6 Settings

The setting page with five tabs for configuration options, including the user profile settings, LLM configuration, email bot, project visibility, and user management.

2.2.6.1 Profile Settings

The first tab of the settings page is the user profile setting, which allows the user to access the profile information, including the profile name, email address, user interface language

and account type. The system implements role-based access control, allowing different permissions depending on the user's role. Administrator users have full access to all features and settings, while project managers have limited, role-specific permissions to managing projects. For the current version of the application, modifying the profile setting is not available through the frontend, which will be improved in later development.

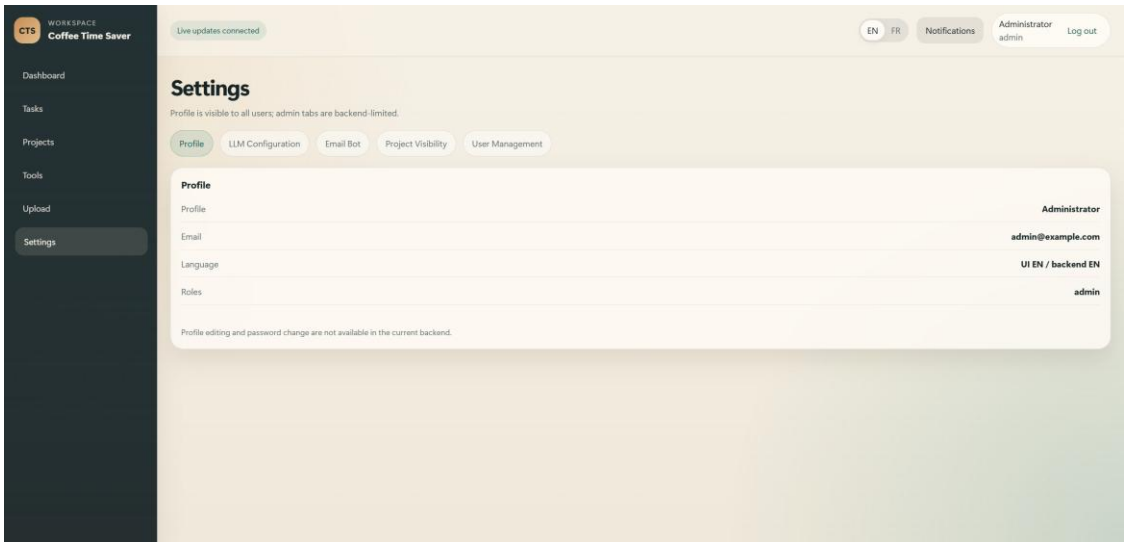

Figure 7 Frontend Prototype: Profile Settings

2.2.6.2 LLM Configuration

The LLM Configuration page displays the list of large language models available in the system, including the model information and API URL. The models can be tested, and their active status can be toggled on the settings page.

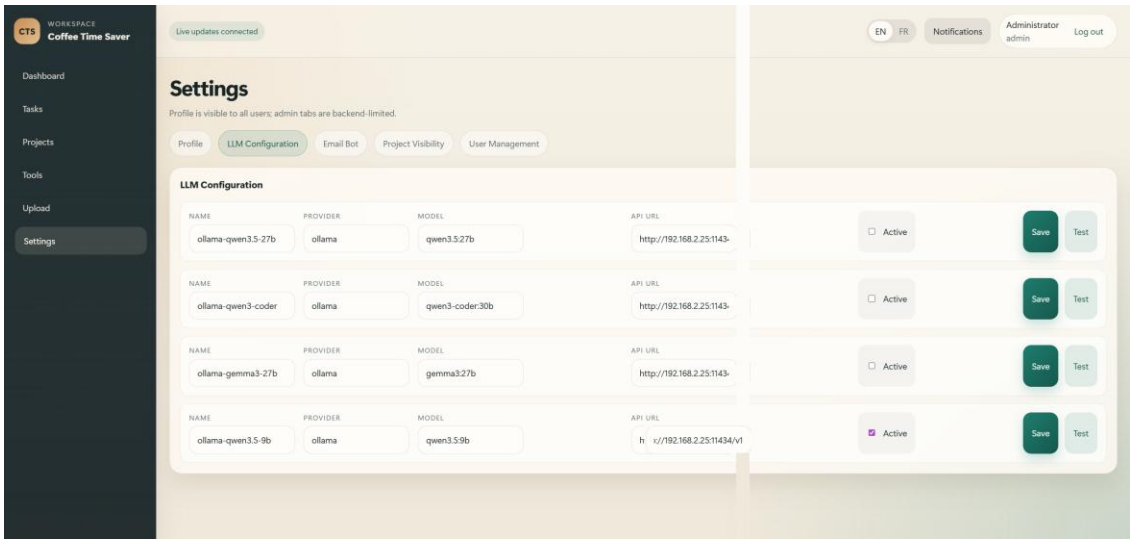

Figure 8 Frontend Prototype: LLM Configuration

2.2.6.3 Email Bot Settings

The Email bot setting page is designed for the admins to manage the email server information, inbox folder and polling interval.

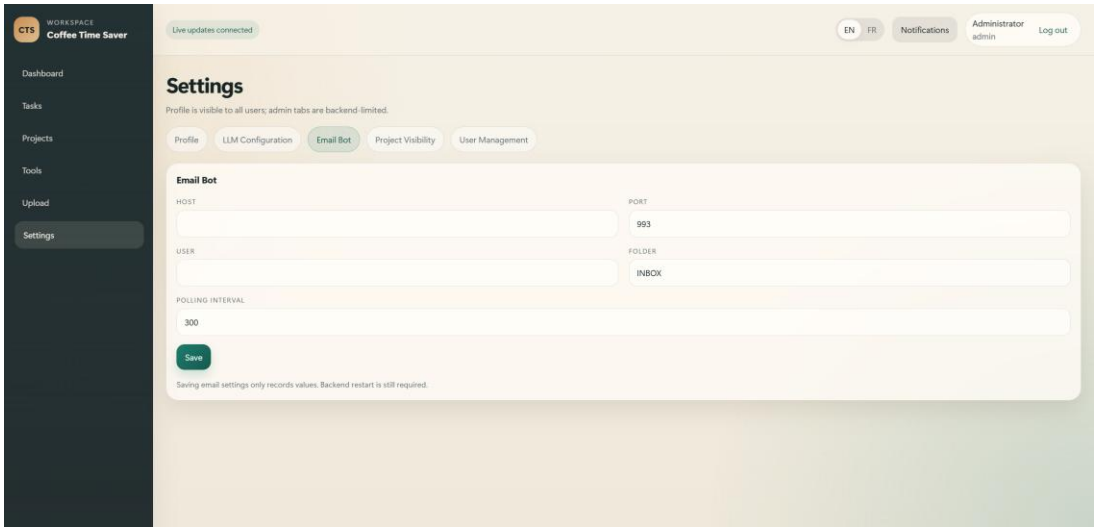

Figure 9 Frontend Prototype: Email Bot Setting

2.2.6.4 Project Visibility

The administrators can manage the project sharing status in the project visibility settings, by toggling the visibility option through the project list.

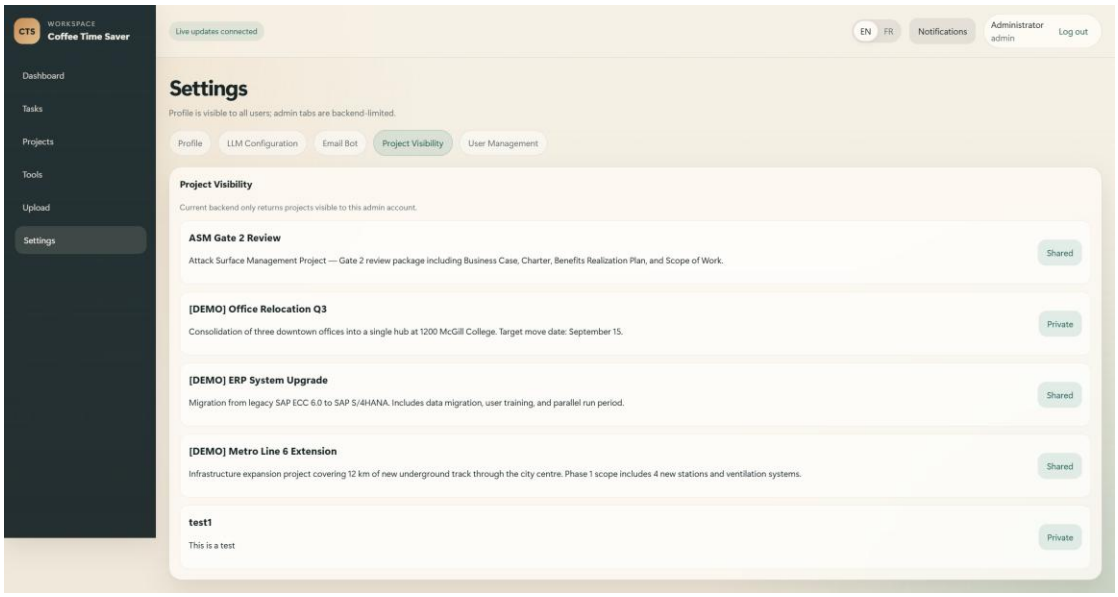

Figure 10 Frontend Prototype: Project Visibility

2.2.6.5 User Management

The role of each account can be assigned in the User Management settings as either administrator or project manager. This allows rapid user management for the administrators.

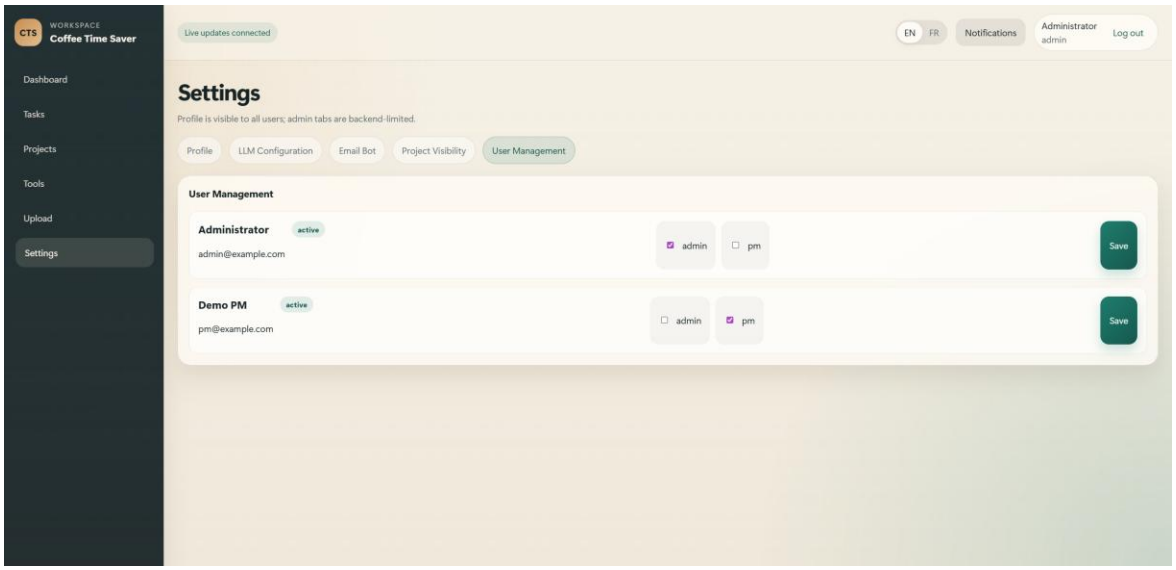

Figure 11 Frontend Prototype: User Management

3. Prototype and Detailed Testing

3.1 Frontend Prototype

This section documents each prototype developed for Coffee Time Saver. For every prototype, we provide its purpose, the requirement or assumption it validates, a description of the implementation, and any modifications made since Deliverable D. Screenshots and diagrams are referenced where applicable.

Prototype 1: Authentication System

Purpose:
Validate JWT-based auth with RBAC can secure all API endpoints and support session refresh.
What it validates:
Requirement for secure multi-role access (admin vs. PM).
Implementation:
backend/core/auth/ (jwt.py, password.py, dependencies.py, providers.py) + frontend/src/api/client.ts (interceptors for token injection and 401 auto-refresh).
Screenshots:
Login page, successful redirect to dashboard, Swagger /docs showing JWT lock icons.

Prototype 2: Dashboard & Daily Briefing

Purpose:
Validate that project data can be aggregated into a single daily view with bilingual support.
What it validates:
Core value proposition "PM gets a consolidated daily view".
Implementation:
backend/modules/dashboard/ + backend/modules/briefing/ + frontend/src/pages/DashboardPage.tsx.
Screenshots:
Dashboard with MetricsGrid, BriefingCard (EN and FR toggle), ActivityFeed.

Prototype 3: Task Management

Purpose:
Validate CRUD + pluggable sort strategy for PM task tracking.
What it validates:
Requirement for centralized task list with priority-based ordering.

Implementation:
backend/modules/tasks/ (service.py, sorter.py) + frontend/src/pages/TasksPage.tsx.
Screenshots:
Task list, create dialog, complete/delete actions, filter controls.

Prototype 4: File Processing Pipeline

Purpose:
Validate end-to-end document upload → parsing → chunking → embedding.
What it validates:
Requirement to ingest project documents and make them searchable/analyzable.
Implementation:
backend/modules/file_processing/ (parsers for PDF, DOCX, XLSX, TXT) + backend/modules/ingestion/ (chunker, language_detect, structurer, embedder) + frontend/src/pages/UploadPage.tsx.
Screenshots:
Upload page with drag-and-drop, progress bars, processing status via WebSocket.

Prototype 5: Project Management

Purpose:
Validate project creation, membership, and access control.
What it validates:
Multi-project support with shared/private visibility.
Implementation:
backend/modules/projects/ + frontend/src/pages/ProjectsPage.tsx.
Screenshots:
My Projects / Shared Projects tabs, create dialog, detail view.

Prototype 6: Settings & LLM Configuration

Purpose:
Validate admin-configurable LLM providers and system settings.
What it validates:
Requirement for flexible LLM backend (OpenAI, Claude, Ollama).
Implementation:
backend/modules/settings/ + backend/modules/llm_gateway/ + frontend/src/pages/SettingsPage.tsx.
Screenshots:
LLM config panel, email bot config, user role management.

Prototype 7: Real-time Updates (WebSocket)

Purpose:
Validate that backend events propagate to frontend in real time.
What it validates:

Requirement for live dashboard/task updates without manual refresh.
Implementation:
backend/core/websocket.py + frontend/src/hooks/useWebSocket.ts + useWsEvent.ts.
Diagram:
Sequence diagram showing: user action → backend write → WebSocket publish → frontend receives event → UI refresh.

Prototype 8: Risk Analyzer Framework (Phase 2 Preview)

Purpose:
Validate the tool registry pattern and demonstrate Phase 2 extensibility.
What it validates:
Dynamic tool discovery and execution pattern.
Implementation:
backend/modules/tools/ (base.py, registry) + backend/modules/tools/risk_analyzer/.
Screenshots:
Tools page showing registry, risk analyzer run/status/report/download flow

3.2 Test Execution and Results

For this stage, we are focusing on building the whole pipeline and testing their functionalities. And most of the tests are based on this purpose. We will do performance testing in the final phase.

3.2.1 Unit Tests

Table 1 unit test

Test ID	Module	Test File	Test escription	Expected	Actual	Status
UT-01	Auth	test_auth.py	Create access token with valid claims	Token contains correct sub/role/type claims	Verified	PASS

Test ID	Module	Test File	Test escription	Expected	Actual	Status
UT-02	Auth	test_auth.py	Access token decoded successfully	Payload matches input	Verified	PASS
UT-03	Auth	test_auth.py	Expired token returns None	None returned	None returned	PASS
UT-04	Auth	test_auth.py	Tampered token returns None	None returned	None returned	PASS
UT-05	Auth	test_auth.py	Refresh token type rejected as access	None returned	None returned	PASS
UT-06	Auth	test_auth.py	Password hash and verify	Hash matches original	Verified	PASS
UT-07	Chunker	test_chunker.py	Chunk short text (< chunk size)	Single chunk returned	1 chunk	PASS
UT-08	Chunker	test_chunker.py	Chunk long text with overlap	Multiple overlapping chunks	Correct count and overlap	PASS
UT-09	Chunker	test_chunker.py	Empty text	Empty list returned	[]	PASS
UT-10	Chunker	test_chunker.py	Exact chunk size boundary	Correct split	Verified	PASS
UT-11	Language	test_language_detect.py	Detect English text	"en"	"en"	PASS
UT-12	Language	test_language_detect.py	Detect French text	"fr"	"fr"	PASS
UT-13–18	Parsers	test_parsers.py	PDF/DOCX/XLSX/TXT extraction	Text extracted correctly	Correct output	PASS

Test ID	Module	Test File	Test escription	Expected	Actual	Status
UT-19–25	Sorter	test_sorter.py	Task priority sorting (various cases)	Tasks ordered by computed score	Correct ordering	PASS
UT-26–31	Structurer	test_structurer.py	Regex-based structured extraction	Structured data fields populated	Correct extraction	PASS

1.1.1 Integration Tests

Table 2 Integration Tests

Test ID	Module	Test Description	Expected	Actual	Status
IT-01	Tasks	Create task via service	Task persisted with UUID PK	Verified	PASS
IT-02	Tasks	List user tasks	Returns all tasks for user	Verified	PASS
IT-03	Tasks	Complete task	status='completed', completed_at set	Verified	PASS
IT-04	Tasks	Delete task (soft)	status='deleted'	Verified	PASS
IT-05	Tasks	Sort tasks by strategy	Order matches hardcoded scoring	Verified	PASS

1.1.2 Frontend Functional Tests

Table 3 Frontend Test

Test ID	Page	Test Description	Expected	Actual	Status
FT-01	Login	Valid credentials → dashboard	Redirect + token stored	Works	PASS
FT-02	Login	Invalid credentials → error	Toast with error message	Works	PASS
FT-03	Dashboard	Load metrics + briefing	Numbers displayed, briefing card rendered	Works	PASS
FT-04	Dashboard	EN/FR language toggle	Briefing content switches language	Works	PASS
FT-05	Tasks	Create new task	Task appears in list	Works (after bug fix)	PASS
FT-06	Tasks	Complete task	Task marked as done	Works	PASS
FT-07	Tasks	Filter by project/priority	List updates locally	Works	PASS
FT-08	Upload	Upload PDF file	File appears with "processing" then "completed" status	Works via WebSocket	PASS
FT-09	Projects	Create project (admin)	Project appears in list	Works	PASS
FT-10	Settings	View LLM config (admin)	Config entries displayed	Works	PASS
FT-11	Tools	Run risk analyzer	Report generated and downloadable	Framework works	PASS
FT-12	WebSocket	Real-time task update	Dashboard refreshes on task creation	Works	PASS

1.1.1 Frontend Functional Tests

Bug ID	Description	Root Cause	Fix Applied	Verified
BUG-01	Task creation returns 500	UUID objects in WebSocket JSON payload	Router converts UUIDs to strings before publish	Yes
BUG-02	Login shows "Network Error"	CORS only allowed port 3000; frontend on 5173	Added localhost:5173 to ALLOWED_ORIGINS in .env	Yes
BUG-03	Frontend .env.example unreliable	localhost DNS resolution inconsistent on some machines	Changed to 127.0.0.1	Yes

3.3 Remaining Work

Table 4 Remainning Work

Item	Purpose	Priority
Daily briefing correctness testing	Validate core value proposition	High
Web UI improvement (layout)	Improve Design Day presentation quality	High
File extraction correctness testing	Ensure ingestion pipeline reliability (Deliverable D ING-04 was a known weakness)	High
Email bot testing	Validate email integration promised in Deliverable D	Medium
LLM analyzer testing	Validate Phase 2 core feature for Design Day demo	High
Writing report	Course deliverable	High
Prepare for presentation	Design Day readiness	High
Wrike plan update	Required by Deliverable E instructions	Medium

4. Updated Test and finalization Plan

4.1 Current Status and Changes Since Deliverable D

Since Deliverable D, the project has transitioned from isolated module validation using a test UI into a fully integrated, production-style web application. The key changes to the project plan are:

Completed since Deliverable D:

Full backend implementation:
All 11 modules (auth, dashboard, briefing, tasks, projects, file processing, ingestion, email bot, LLM gateway, settings, tools) are implemented and passing unit tests (31 passed).

Database schema finalized:
Alembic migration creates all 14 tables with pgvector support; seed script provides demo data.

New production frontend:
React 19 + TypeScript + Vite replaced the Deliverable D test UI. All 7 pages (login, dashboard, tasks, projects, upload, tools, settings) are functional.

Integration bugs resolved:
UUID serialization in WebSocket events (BUG-01), CORS configuration for Vite port (BUG-02), and environment variable reliability (BUG-03).

Docker Compose deployment:
all 6 services (frontend, backend, db, redis, celery-worker, celery-beat) are defined and operational.

What changed from the Deliverable D plan:

In Deliverable D (§4.2), we outlined five steps:

(1) client feedback via test UI, (2) module logic revision, (3) email bot integration, (4) build new UI, (5) full pipeline test. Steps 1-4 are complete. Step 5, full pipeline correctness testing, is the focus of the remaining plan below.

4.2 Updated Project Plan

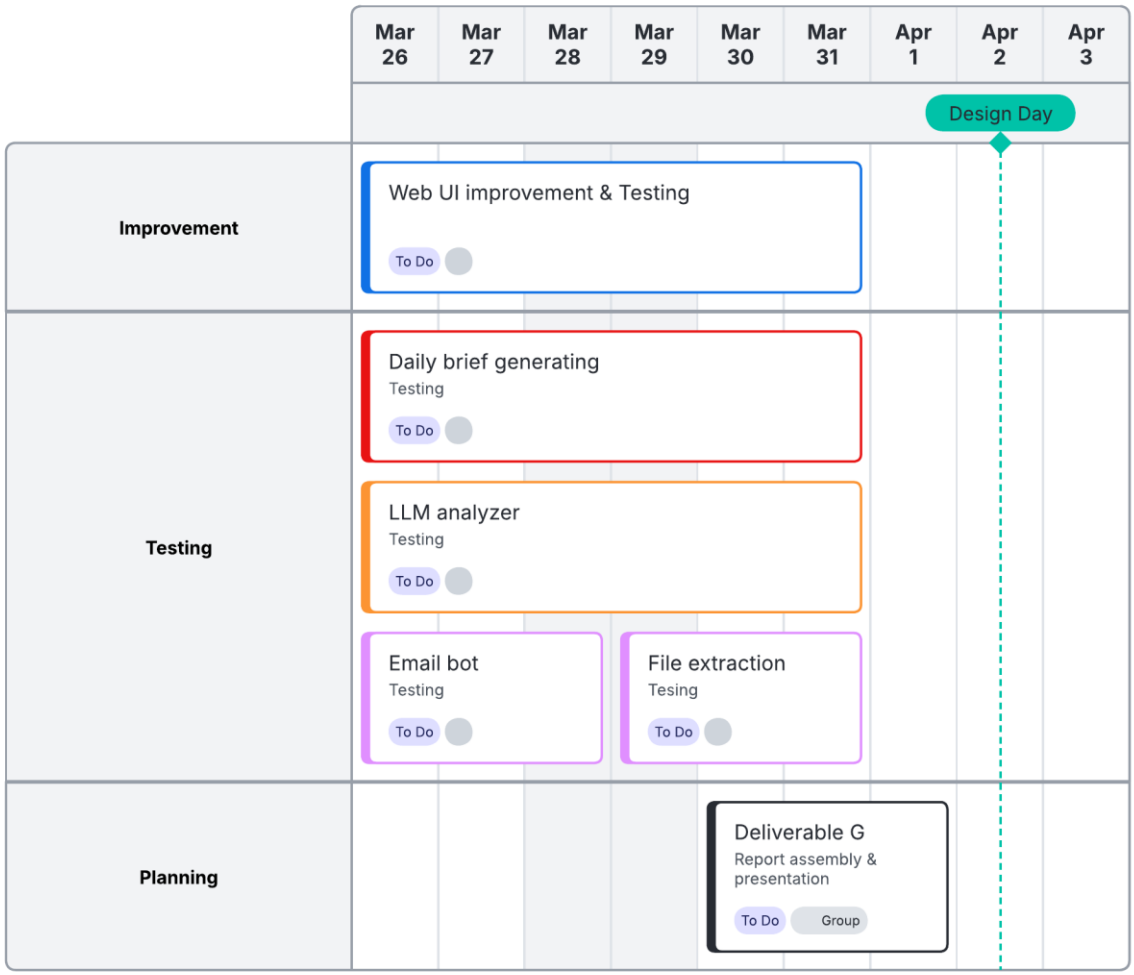

Figure 12 Gantt Chart

4.3 Task Assignment

Table 5 Task Assignment

Member	Assigned Tasks
Zhenyan Xing	Daily brief generating testing, Analyzer testing
Zhe Wang	Email bot testing

Member	Assigned Tasks
Aoqi Liu	Web UI improvement & testing, Design Day demo script
Lizzy-Preiz	File Extraction testing, Report assembly, presentation prep

4.4 Contingency Plans

1.1.2 Design Day Presentation Risks

Likelihood	Risk	Description	Backup Plan
High	LLM Inference Too Slow / Timeout	LLM responds slowly or hangs during live demo	(1) Prepare slides comparing human analysis vs. LLM output side-by-side, so the demo can continue without waiting for inference. (2) Pre-cache LLM outputs for key demo scenarios (risk analysis report, daily briefing) so results can be shown instantly if the live call stalls.
Medium	Pitch Exceeds 3-Minute Limit	Sections run long, demo gets cut short or interrupted	(1) Rehearse at least 2 times with strict timing. (2) Assign a timekeeper role to one member (Lizzy-Preiz) to signal transitions.
Low	Member Absent / Equipment Failure / Function Unsolvable Failure	Someone cannot attend, or mic/camera fails on the day	(1) Each member learns at least one other member's section so any two people can cover the full demo. (2) If a module fails live, downgrade to showing the concept via screenshots/slides. (3) Keep a backup laptop ready.
Very Low	Server Down / Network Disconnected	Cannot connect to hosted server during live demo	(1) Record a full demo video as backup beforehand. (2) If live demo fails, immediately switch to the recorded video. (3) Test server connectivity 30 minutes before the presentation.

1.1.1 Development & Testing Risks

Likelihood	Risk	Description	Backup Plan
High	LLM provider unavailable during testing	Cannot complete LLM analyzer or briefing correctness testing	Fall back to Ollama local model (Qwen3.5-27B on local GPU); if local model also fails, present the execution framework with template-based fallback results and document the limitation.
Medium	IMAP test mailbox configuration issues	Email bot testing blocked	Use mock email data already seeded in the database via seed_demo.py; document email bot as "tested with seeded data, live IMAP validation pending."
Medium	File extraction fails on real SSC documents	Cannot demonstrate ingestion accuracy on anonymized data	Use the test documents already successfully processed in test_data/; focus demo on the documents that parsed correctly and document extraction issues as known limitations.
Low	Report writing takes longer than planned	Late submission risk	Start report sections in parallel with testing — Lizzy-Preiz begins Section 4 while testing is still running; Aoqi Liu prepares screenshots concurrently with UI polish.

5. Conclusions and Recommendations for Future Work

1.1.2 Status conclusion

For the next delivery until the design day, we will focus on testing and improving the current core services including daily brief generating and LLM analyzer, as well as important functionalities such as the email forwarding feature and file extraction tools. On the other hand,

we will also refine the current implementation of the user interface based on testing results and client feedback.

5.1 Recommendations for Future Work

Sustainability and scalability. Deliverable D identified that repeated analysis on unchanged documents wastes compute and energy. A caching layer with content-hash change detection should be added so that the ingestion pipeline and LLM analysis skip reprocessing when the source has not changed. On the scalability side, the current Ollama backend does not handle concurrent multi-user inference well. Migrating to vLLM for parallel serving, or switching to a cloud LLM provider API for production, would resolve this bottleneck. The JWT authentication system should also be upgraded to OAuth 2.0 (e.g., Microsoft Entra ID) to integrate with SSC's existing identity infrastructure.

Usability. Users currently see only a binary "processing / completed" status for file uploads and LLM analysis. Adding a progress bar or estimated time indicator for long-running tasks — particularly risk analysis, which involves multiple sequential LLM calls — would significantly improve the experience. Additionally, profile editing, password change, and project-level detail panels (documents, timeline, risk indicators) remain as placeholders and should be completed to deliver a full self-service workflow.

LLM quality improvements. Deliverable D testing revealed three unresolved issues that carry into future work: (1) the Drift Detection module fails the same three edge cases (DFT-03, DFT-05, DFT-06) across all LLM tiers, which requires adding few-shot examples and bilingual SOW baselines; (2) the Priority Ranker hallucinates justifications (PRI-05), which can be mitigated by

adding a grounding instruction to the prompt; and (3) the Risk Analyzer needs a formal accuracy benchmark comparing AI-generated risk reports against a human expert baseline to build client confidence.

Local anonymization model. As noted in the Deliverable D contingency plan, a local LLM with anonymization capabilities has become available. Given SSC's strict data residency requirements, evaluating this model as a provider option in the LLM Gateway could open deployment possibilities that cloud-only providers cannot support. This evaluation should compare extraction quality, anonymization reliability, latency and integration effort.

6. Bibliography

[1] S. Ramírez, "FastAPI," 2024. [Online]. Available: https://fastapi.tiangolo.com/
[2] Meta Platforms, Inc., "React Documentation," 2024. [Online]. Available: https://react.dev/
[3] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/
[4] A. Krosing, "pgvector: Open-source vector similarity search for Postgres," GitHub, 2024. [Online]. Available: https://github.com/pgvector/pgvector
[5] Ask Solem et al., "Celery: Distributed Task Queue Documentation," 2024. [Online]. Available: https://docs.celeryq.dev/
[6] M. Bayer, "SQLAlchemy 2.0 Documentation," 2024. [Online]. Available: https://docs.sqlalchemy.org/
[7] P. Mendrs, "Zustand: Bear necessities for state management in React," GitHub, 2024. [Online]. Available: https://github.com/pmndrs/zustand
[8] E. You et al., "Vite: Next Generation Frontend Tooling," 2024. [Online]. Available: https://vitejs.dev/
[9] i18next Contributors, "react-i18next Documentation," 2024. [Online]. Available: https://react.i18next.com/
[10] Redis Ltd., "Redis Documentation," 2024. [Online]. Available: https://redis.io/docs/
[11] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force (IETF), May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519
[12] Docker Inc., "Docker Compose Documentation," 2024. [Online]. Available: https://docs.docker.com/compose/
[13] Government of Canada, "2024-25 Departmental Plan: Raison d'être, mandate and role," Shared Services Canada, 2024. [Online]. Available: https://www.canada.ca/en/shared-services/corporate/about-us/transparency/departmental-plan/2024-25/2024-25-departmental-plan/raison-etre-mandate-role.html
[14] Government of Canada, "Canada's Digital Ambition 2024-25," Treasury Board of Canada Secretariat, 2024. [Online]. Available: https://www.canada.ca/en/government/system/digital-government/canada-digital-ambition/canada-digital-ambition-2024-25.html
[15] Government of Canada, "Canada's Digital Ambition 2023-24," Treasury Board of Canada Secretariat, 2023. [Online]. Available: https://www.canada.ca/en/government/system/digital-government/canada-digital-ambition/canada-digital-ambition-2023-24.html
[16] Shared Services Canada, "Project Management & Delivery Branch Overview," Internal Documentation, 2026.
[17] Shared Services Canada, "Project Governance and Operating Framework (PGOF)," Version 2025-03, 2025.